

Intelligent Adaptive Traffic Light Control System

Ksenia Polyantseva

Moscow Technical University of Communications and
Informatics
Moscow, Russia
k.a.polyantseva@mtuci.ru

Ernest Latypov

Moscow Technical University of Communications and
Informatics
Moscow, Russia
e.m.latypov@edu.mtu.ru

Abstract—This article is devoted to the development of an intelligent traffic light control system. The main goal is to research and create an adaptive system based on machine learning and computer vision methods to increase the capacity of intersections and reduce waiting times for both vehicles and pedestrians. As part of the work, a platform was designed and implemented that uses the SUMO simulator to simulate the urban transport network and the DQN (Deep Q-Network) algorithm to train a traffic light control agent. To collect data on the current traffic situation, two YOLOv11 models were integrated: one for vehicle detection, the other for pedestrian recognition. The study includes the results of testing and comparing the two models.: with static and adaptive controls.

Keywords—intelligent transport systems; traffic light control; reinforcement learning; DQN; YOLO; SUMO; RNN; computer vision

I. INTRODUCTION

In the context of the rapid growth of motorization [1] and the constant increase in the burden on urban transport infrastructure, the problem of effective traffic management is becoming critically important. Traditional approaches based on fixed time cycles of traffic lights are not able to respond to dynamic changes in traffic flows, which leads to congestion, increased travel time, increased economic costs [2] and environmental degradation. In this regard, various technologies of intelligent traffic management systems (ISMS) have been developed and implemented in recent decades. One of the first successful examples were the SCATS (Sydney Coordinated Adaptive Traffic System) and SCOOT (Split, Cycle, Offset Optimization Technique) systems [3], which are still used in many cities around the world. SCOOT, for example, analyzes data from induction loops embedded in the roadway and dynamically adjusts the duration of traffic light phases, which reduces the average waiting time for transport. More modern centralized platforms, such as the British UTMC (Urban Traffic Management and Control) [4], combine data from surveillance cameras, GPS trackers and other sensors, ensuring coordination of traffic lights at the city level. The projects implemented in Singapore and China are particularly impressive: the ERP (Electronic Road Pricing) system in Singapore combines dynamic pricing for entering the city center with adaptive traffic light control [5], and the Chinese ET City Brain project from Alibaba [6] uses artificial intelligence to analyze video streams from thousands of cameras, allowing not only to optimize the operation of traffic

lights, but also to predict accidents, which in Hangzhou led to a 15% increase in traffic speed. Active developments are also underway in Russia: a network of more than a thousand smart intersections operates in Moscow [7], and pilot projects using VOCORD Traffic complexes for data collection and management have been launched in St. Petersburg [8] and other cities [9].

However, even these advanced systems often rely on predefined algorithms or statistical models, which limits their ability to learn and adapt in real time. The real breakthrough is the application of machine learning methods [10], in particular, Reinforcement Learning [11], which allows the system not only to follow predefined rules, but to learn independently from experience, maximizing the effectiveness of management through trial and error [12]. That is why the purpose of this work is to research and develop an intelligent traffic light control system based on the DQN (Deep Q-Network) algorithm [13] and the integration of YOLOv11 computer vision models [14], capable of not only adapting to the current traffic situation, but also continuously improving its strategy to achieve the goal of minimizing waiting time as much as possible. for both vehicles and pedestrians.

II. THEORETICAL FOUNDATIONS AND PLATFORM DESIGN

The projected platform is based on the SUMO (Simulation of Urban MObility) traffic simulator [15], which serves as a virtual environment for modeling the urban road network, including vehicles, pedestrians, road infrastructure and traffic lights. The interaction between the simulator and the control agent is carried out through the TraCI (Traffic Control Interface), which provides two-way data exchange in real time, allowing an external algorithm to read the state of the environment and send control commands. The architecture of the system is strictly divided into three main layers: the simulation level, the perception level, and the decision-making level. At the simulation level, traffic flows and pedestrians are generated, traffic control is performed, and data is collected. Data is collected at each entrance to the intersection (EB, SB, WB, NB) and for each lane, forming a discretized vector of the state of the environment, which corresponds to the approach described in [16]:

$$s_t = [q_1^{\text{car}}, q_2^{\text{car}}, \dots, q_n^{\text{car}}, p_1^{\text{ped}}, p_2^{\text{ped}}, \dots, p_m^{\text{ped}}] \quad (1)$$

where q_i^{car} - the number of cars on the i-th lane, p_j^{ped} - the number of pedestrians on the j-th crossing.

To account for traffic, a computer vision module is integrated into the system, which uses two pre-trained YOLOv11 models: 'yolov11_vehicle_detector' to determine the number of vehicles and 'yolov11_person_detector' to count the number of pedestrians in the pedestrian crossing area for the sake of joint optimization of traffic and pedestrian flows [17].

The core of the system is an agent implemented based on the Deep Q-Network (DQN) algorithm. His task is to choose the best possible action (to continue the phase or switch it):

$$Q(s_t, a_t; \theta) \rightarrow R \quad (2)$$

where s_t - the current state of the environment, a_t - the agent's action, and θ - the parameters of the neural network model. The agent strives to choose an action a_t that maximizes the expected reward:

$$a_t = \arg \max_a Q(s_t, a; \theta) \quad (3)$$

where argmax is the operation of choosing an action that maximizes the value of the Q-function.

The reward for the action was formed as a negative value proportional to the total waiting time for cars and pedestrians, plus a positive bonus for increasing the capacity of the intersection:

$$r_t = -w_{car} \cdot T_{car}^{wait} - w_{ped} \cdot T_{ped}^{wait} + b \cdot \Delta N_{passed} \quad (4)$$

where T_{car}^{wait} and T_{ped}^{wait} are the total waiting time for cars and pedestrians, respectively, ΔN_{passed} is the increase in the number of objects passing through the intersection, w_{car} and w_{ped} - weights, and b is a bandwidth bonus.

This reward formula encourages the agent not only to minimize the waiting time, but also to maximize the flow of traffic through the intersection.

The input layer of the agent's neural network accepts a state vector and consists of 14 neurons (12 for traffic lane data, 1 for pedestrians, 1 for the current phase of the traffic light); two hidden layers contain 64 neurons each with a nonlinear activation function ReLU, which allows the network to effectively model complex nonlinear dependencies. The output layer consists of two neurons, each of which outputs a Q-value estimate for one of two possible actions. To stabilize the learning process in DQN, two key mechanisms are used: the Replay Buffer and the Target Network. The experience buffer is a cyclic queue of a fixed size (for example, 10,000

transitions), in which tuples (state, action, reward, next_state, done) are stored. At each training step, a mini-packet of data is randomly selected from the buffer, which ensures a more stable update of the weights.

The target network was also used in the work - a copy of the main one, the parameters of which are updated with a delay. This allows you to use stable target values when calculating the loss function. The parameters of the main network are updated by gradient descent, minimizing the Huber Loss function, which is more resistant to outliers than MSE:

$$L(\theta) = E_{(s, \alpha, r, s') \sim U(D)} \left[\left(r + \gamma \max_{\alpha'} Q(s', \alpha'; \theta^-) - Q(s, \alpha; \theta) \right)^2 \right] \quad (5)$$

where D is the buffer of experience, U(D) is the uniform distribution over the buffer, γ is the discount coefficient, θ^- are the parameters of the target network.

The parameters of the target network are updated at a specified frequency (for example, every 100 steps) by copying weights from the main network. The e-greedy strategy is used to balance the exploration of a new strategy and the exploitation of an already found optimal one. The probability of choosing a random action decreases exponentially over the course of episodes:

$$\epsilon_t = \epsilon_{min} + (\epsilon_{start} - \epsilon_{min}) \cdot e^{-\beta t} \quad (6)$$

where β is the rate of decrease of ϵ , t is the episode number.

This strategy allows the agent to actively explore the action space at the beginning of training, and as experience accumulates, he increasingly chooses an action with the maximum Q-value.

The entire system is implemented in Python 3 using TensorFlow/Keras libraries for building and training a neural network, NumPy and Pandas for data processing and analysis, as well as Matplotlib for visualizing learning outcomes and performance metrics. The configuration of the road network and routes is set in XML files ('grid.net.xml', 'all.rou.xml'), which provides a high degree of customization and the ability to quickly adapt the platform to various scenarios of urban development. Thus, the proposed architecture represents a complete and self-sufficient cycle: from collecting data on the current state of the environment through sensors and cameras, to processing this data with a neural network model, making a decision and sending a control action back to the simulator, which creates the basis for creating a fully autonomous and adaptive traffic management system. [18]

III. SYSTEM IMPLEMENTATION

The implementation began with the preparatory stage, at which a virtual transport network was created. Using the

netconvert tool and an XML description, a typical urban intersection system was modeled with four main directions, each of which included three lanes (Fig.1). To ensure the realism of the scenario, a comprehensive traffic flow was created: using the utility 'randomTrips.py' routes have been generated for cars (200 streams), buses, trucks, motorcycles, and even emergency services that can be assigned priority travel in SUMO. Separately, using the '--pedestrians' flag, a pedestrian stream (150 streams) was generated, simulating waves of people crossing the road during rush hours. All routes have been combined into a single file 'all.rou.xml' with the help of 'duarouter', which allowed us to create a rich and dynamic testing environment.

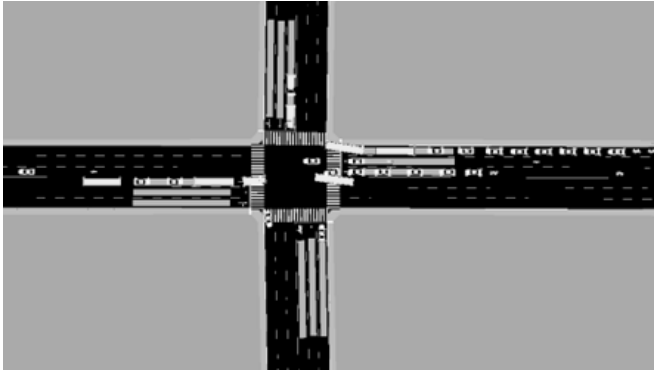


Fig. 1. The model of the road network.

The key stage was the creation of a DQN agent. The implementation was done in a file `dqn_agent.py`. The neural network was built using Keras: a sequential model with an input layer of 14 neurons, two hidden layers of 64 neurons each with ReLU activation, and an output layer of 2 neurons. Huber was chosen as the loss function, and Adam with a learning rate of 0.001 was chosen as the optimizer. To stabilize learning, a target network was implemented, the parameters of which were copied from the main network every 100 steps. The ReplayBuffer experience buffer was implemented based on the deque framework with a maximum capacity of 10,000 transitions. Training was started only after accumulating 1000 transitions, and at each step, a 32-bit mini-batch was randomly selected from the buffer to update the weights. To prevent instability, the gradients were limited to a maximum value of 1.0.

Special attention was paid to the integration of computer vision. The training was conducted using 100 epochs (Fig.2, Fig.3) using augmentation (rotations, brightness changes, noise). The final metrics of the models are: for transport - $mAP50-95 = 0.78$, for pedestrians - $mAP50-95 = 0.83$. Module `detection.py` simulates the operation of cameras: for each direction (EB, SB, WB, NB), a static image is loaded (Fig.4), which is programmatically cut into three vertical segments (stripes). Each segment is fed to the `yolov11_vehicle_detector` input for vehicle counting. A separate image of the pedestrian crossing area is analyzed by the `yolov11_person_detector` model (Fig.5). The resulting numbers (12 values for lanes + 1 for pedestrians), together with the current phase of the traffic light, form a 14-dimensional state vector, which is fed to the input of the DQN agent.

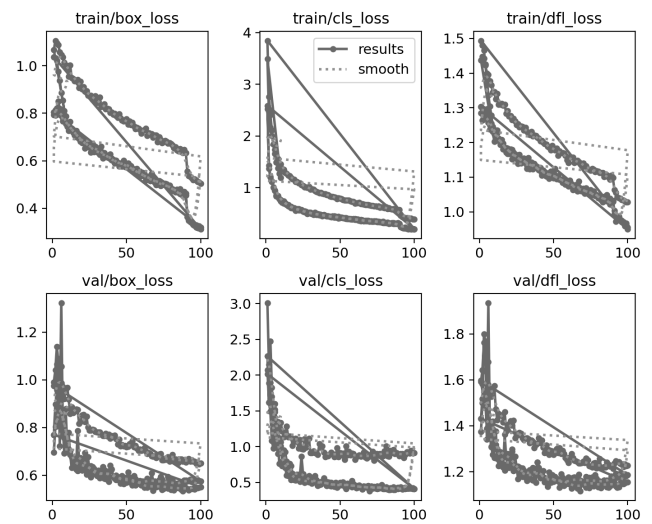


Fig. 2. Training the 'yolov11_vehicle_detector' model.

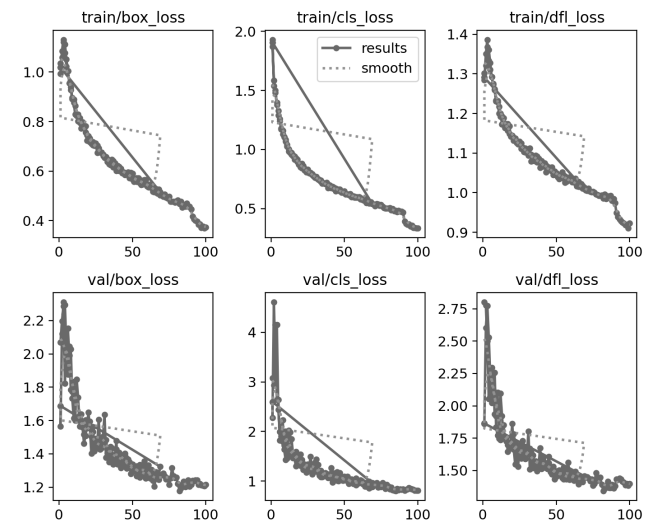


Fig. 3. Training the 'yolov11_person_detector' model.



Fig. 4. Example of transport detection.



Fig. 5. Example of pedestrian detection.

The main cycle of the system is implemented in the script main.py. He manages 100 training episodes. The simulation steps can be described as follows: through TraCI, raw data is extracted from SUMO sensors, then the module detection.py processes images and returns updated data about vehicles and pedestrians, the agent selects an action: "continue phase" or "switch phase", after which the command is sent back to SUMO via TraCI. The simulator performs a step by updating the state of the environment, and after that the reward is calculated based on the waiting time and bandwidth, and the transition (s, a, r, s', done) is saved in the ReplayBuffer - if there are >1000 transitions in the buffer, the training step is started: a mini-batch is selected from the buffer, target Q-values are calculated. The values using the target network and the weights of the main network are updated.

IV. COMPARISON OF TRAFFIC LIGHT CONTROL MODELS

To assess the effectiveness of the adaptive system, it was compared with the basic model in which traffic lights were controlled on a fixed cycle. For the sake of visual demonstration of the results, graphs were constructed after completion of the training, characterizing the dynamics of traffic, the number of waiting pedestrians and cars, as well as the progress of agent training (Fig.6, Fig.8, Fig.10, Fig.12, Fig.14). In addition, graphs were constructed for the static system to compare two logics (Fig.7, Fig.9, Fig.11).

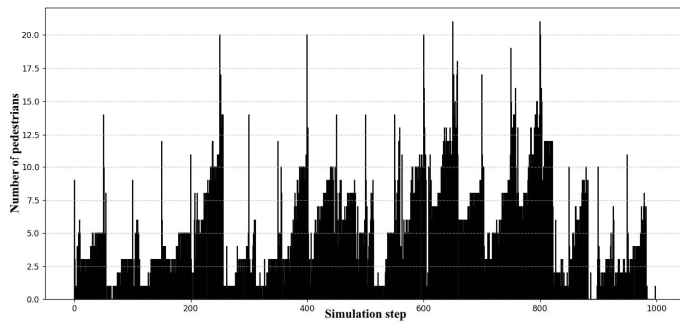


Fig. 6. The number of pedestrians by steps in an adaptive control simulation.

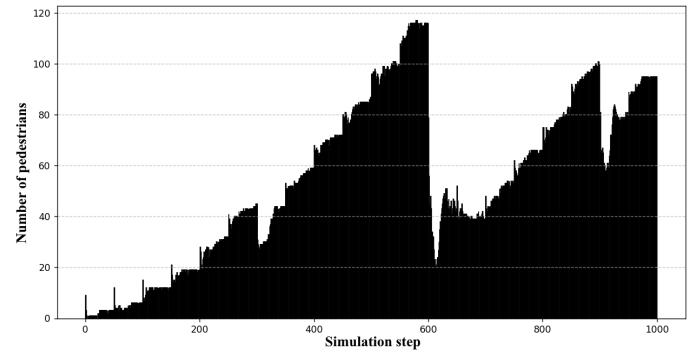


Fig. 7. The number of pedestrians by steps in a simulation with static control.

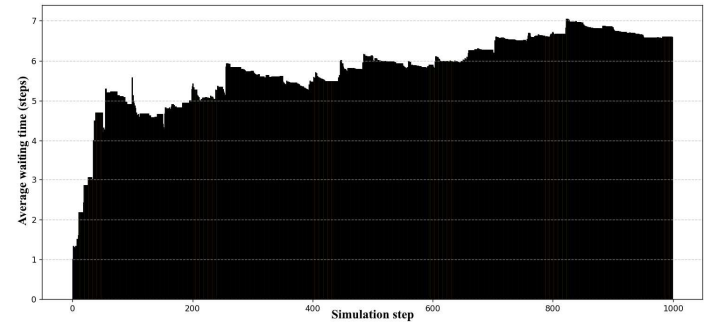


Fig. 8. Average waiting time for pedestrians in an adaptive control simulation.

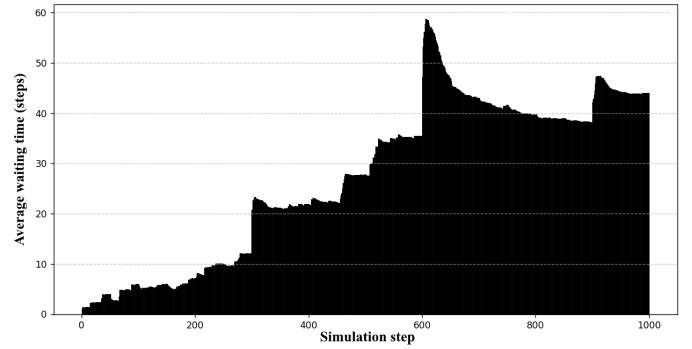


Fig. 9. Average waiting time for pedestrians in a static control simulation.

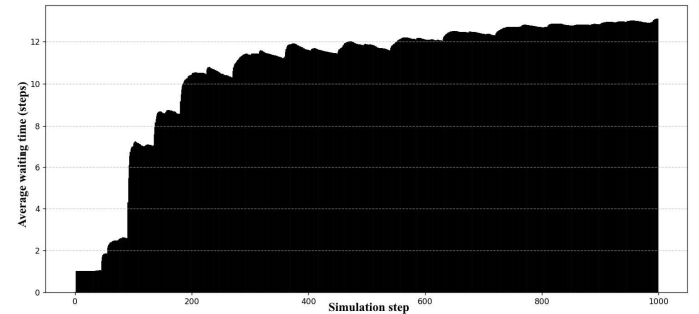


Fig. 10. Average waiting time for transport in an adaptive control simulation.

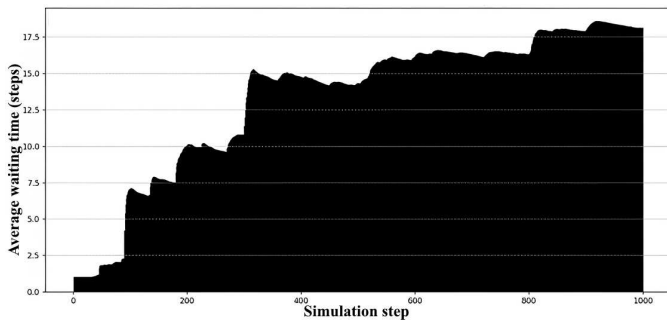


Fig. 11. Average waiting time for a transport in a simulation with static control.

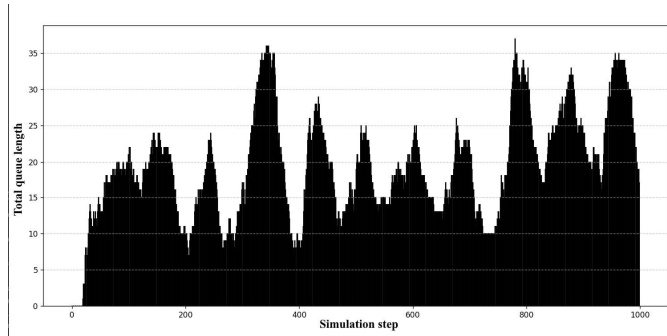


Fig. 12. Total length of a car queue in an adaptive control simulation.

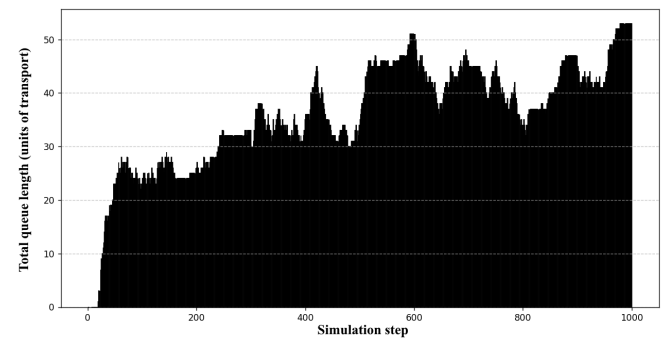


Fig. 13. The total length of the car queue in the simulation with static control.

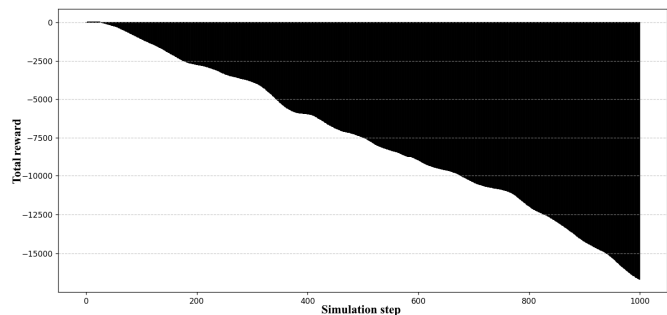


Fig. 14. Total reward.

From the analysis of the graphs shown in Fig. 6-14, it can be concluded that the created adaptive traffic light control system is effective. As can be seen from the figures, when

using the trained model, the waiting time for pedestrians and cars is significantly reduced compared to the fixed signal switching time. The waiting time for transport and pedestrians is reduced by almost 30% on average, and the total length of the queue of cars is also reduced. In addition, the cumulative remuneration received by the agent during the training increases, which indicates the formation of an optimal management strategy. This confirms the fact that the use of machine learning algorithms, in this case DQN, allows for the development of more sensitive and efficient traffic management systems that can be adapted to respond to dynamic situations in real time. The average results are shown in table 1.

TABLE I. COMPARING THE METRICS OF THE TWO MODELS

Metric	Fixed control	Adaptive control
Average waiting time for pedestrians	45 steps	25 steps
Average waiting time for a vehicle	60 steps	42 steps
Total queue length	180 m	120 m
Throughput capacity	50 vehicle/min	65 vehicle/min

CONCLUSION

In the course of the work, an intelligent adaptive traffic light control system based on the DQN algorithm and YOLOv11 computer vision models was successfully developed and tested. The system dynamically adjusts the phases of the traffic light, analyzing traffic and pedestrian flows in real time, which significantly improved key indicators: waiting time for pedestrians decreased by 44%, transport - by 30%, and the total length of queues decreased by 33%. This confirms the high efficiency of the proposed approach in comparison with traditional static systems. The developed platform demonstrates the practical applicability of machine learning methods for optimizing urban traffic and can serve as a basis for implementation into real infrastructure within the framework of the "smart city" concept.

REFERENCES

- [1] The World Bank, "Sustainable mobility for the 21st century," The World Bank, Washington, DC, USA. Jul. 10, 2017. Accessed: Jun. 03, 2025. [Online]. Available: <https://www.worldbank.org/en/news/feature/2017/07/10/sustainable-mobility-for-the-21st-century>.
- [2] V. A. Markichev, "Assessment of losses associated with traffic jams in a metropolis", *Young Scientist*, no. 22 (126), part 2, pp. 17-19, 2016. Accessed: Jun. 03, 2025. [Online]. Available: <https://moluch.ru/archive/126/33660/>.
- [3] A. Stevanovic and P. T. Martin, "Split-cycle offset optimization technique and coordinated actuated traffic control evaluated through microsimulation," *Transportation Research Record*, vol. 2080, no. 1, pp. 48-56, 2008, doi: 10.3141/2080-06.
- [4] CIHT, "Urban Traffic Management and Control Systems," *Chartered Institution of Highways & Transportation*, London, UK. 2015. Accessed: Jun. 03, 2025. [Online]. Available: https://www.ciht.org.uk/media/4367/urban_traffic_management_and_control_systems.pdf.
- [5] Land Transport Authority (LTA), "Electronic Road Pricing (ERP)," OneMotoring, Singapore. Accessed: Jun. 01, 2025. [Online]. Available: <https://onemotoring.lta.gov.sg/content/onemotoring/home/driving/ERP/ERP.html>.

- [6] Alibaba Cloud, "How ET City Brain Is Transforming the Way We Live, One City at a Time," Alibaba Cloud. Accessed: Jun. 01, 2025. [Online]. Available: https://www.alibabacloud.com/blog/how-et-city-brain-is-transforming-the-way-we-live-one-city-at-a-time_593745.
- [7] M. Gorodnichev and M. Moseva, "On the Problem of Developing a Fault-Tolerant High-Loaded Cluster of Support for an Intelligent Transportation System," in *2023 Wave Electronics and its Application in Information and Telecommunication Systems (WECONF)*, St. Petersburg, Russian Federation, 2023, pp. 1-6, doi: 10.1109/WECONF57201.2023.10148007.
- [8] E. Tourouta, M. Gorodnichev, K. Polyantseva, and M. Moseva, "Providing Fault Tolerance of Cluster Computing Systems Based on Fault-Tolerant Dynamic Computation Planning," in *Digitalization of Society, Economics and Management*, E. Zaramenskikh and A. Fedorova, Eds. Cham, Switzerland: Springer, 2022, pp. 119-129, vol. 53, Lecture Notes in Information Systems and Organisation, doi: 10.1007/978-3-030-94252-6_10.
- [9] K. A. Polyantseva, M. G. Gorodnichev, M. S. Moseva, and T. D. Potapchenko, "On the Applicability of Neural Networks in the Tasks of Detecting Dangerous Movement," in *2019 Wave Electronics and its Application in Information and Telecommunication Systems (WECONF)*, St. Petersburg, Russia, 2019, pp. 1-4, doi: 10.1109/WECONF.2019.8840654.
- [10] German Aerospace Center (DLR), "Simulation of Urban Mobility (SUMO) Documentation," *Institute of Transportation Systems*, Berlin, Germany. Accessed: Jun. 01, 2025. [Online]. Available: <https://sumo.dlr.de/docs/index.html>.
- [11] S. M. A. Shabir, M. U. Khalid, A. R. Javed, M. M. M. Alhaisoni, T. R. Gadekallu, and Z. Jalil, "A comprehensive survey of digital twins for smart cities: Architecture, security, and applications," *Information Sciences*, vol. 653, Jan. 2024, Art. no. 119484, doi: 10.1016/j.ins.2023.119484.
- [12] C. Chen, H. Wei, N. Xu, G. Zheng, M. Yang, Y. Xiong, K. Xu, and Z. Li, "Toward A Thousand Lights: Decentralized Deep Reinforcement Learning for Large-Scale Traffic Signal Control," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020, vol. 34, no. 04, pp. 3414-3421, doi: 10.1609/aaai.v34i04.5744.
- [13] V. Mnih et al., "Playing Atari with Deep Reinforcement Learning," DeepMind Technologies, London, UK, Tech. Rep., Dec. 2013. Accessed: Jun. 21, 2025. [Online]. Available: <https://arxiv.org/pdf/1312.5602>.
- [14] M. S. Moseva, M. G. Gorodnichev, K. A. Polyantseva, A. V. Sheremetev, and K. A. Dzhabrailov, "Development of a Platform for Road Infrastructure Digital Certification," in *2021 Intelligent Technologies and Electronic Devices in Vehicle and Road Transport Complex (TIRVED)*, Moscow, Russian Federation, 2021, pp. 1-8, doi: 10.1109/TIRVED53476.2021.9639102.
- [15] T. Fatkhulin, Y. Leokhin, A. Zanegin, and A. Rakhmatova, "Development and Research of a Modified Gradient Boosting Method Effectiveness to Solve Applied Problems of Time-Series Forecasting," in *2025 Systems of Signals Generating and Processing in the Field of on Board Communications*, Moscow, Russian Federation, 2025, pp. 1-10, doi: 10.1109/IEEECONF64229.2025.10948023.
- [16] N. Deshpande and A. Spalanzani, "Deep Reinforcement Learning based Vehicle Navigation amongst pedestrians using a Grid-based state representation," in *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*, Auckland, New Zealand, 2019, pp. 2081-2086, doi: 10.1109/ITSC.2019.8917299.
- [17] B. Poudel, X. Wang, W. Li, L. Zhu, and K. Heaslip, "Joint Pedestrian and Vehicle Traffic Optimization in Urban Environments using Reinforcement Learning," Preprint arXiv, p. 2504.05018, 2025. Accessed: Jun. 21, 2025. [Online]. Available: <https://arxiv.org/abs/2504.05018>.
- [18] Y. Leokhin, T. Fatkhulin, and K. Boitsov, "Computer Vision Methods in Applied Problems of Classifying Objects in Images," in *2025 Wave Electronics and its Application in Information and Telecommunication Systems (WECONF)*, St. Petersburg, Russian Federation, 2025, pp. 1-10, doi: 10.1109/WECONF65186.2025.11017109.